

- 现在我有了完整的代码，来回答你的问题。

同步还是异步，看 SOC 侧用什么 API 发送

关键规则：同步/异步不取决于 topic 配置，取决于 SOC 侧调用时用的是哪个发送 API。

判断方法：一行代码就能看出来

```
// ===== 异步 =====
dcms_mcu_topic_send(topic_id, data, len);
// 特点: 返回 int32_t (成功/失败), 不阻塞, 立即返回
// 响应通过 resp_cbk 异步回调

// ===== 同步 =====
dcms_mcu_topic_send_sync(topic_id, data, len, rsp, &rsp_len, timeout_us);
// 特点: 有 rsp buffer 参数, 有 timeout_us 参数
// 调用会阻塞, 直到收到响应或超时

// ===== 同步 (更上层) =====
minidcos_service_call(service_name, &in, &out, retry, timeout_us);
// 特点: 有 out buffer 参数, 有 timeout_us 参数
// 阻塞等响应, out 里直接拿到返回值
```

你图表里的两个服务 — 都是异步

追代码确认。两个服务的 SOC 侧调用：

DCMS_TOPIC_CALIB_PRECONDITION_CHECK — DiagCalib_Dynamic.c:244：

```
// * 用的是 dcms_mcu_topic_send() — 异步！
retDcms = dcms_mcu_topic_send(DCMS_TOPIC_CALIB_PRECONDITION_CHECK,
                               (uint8_t*)&req, sizeof(req));
// 立即返回，不等待
```

DCMS_CLIENT_DMM_CALIB_MODE_CMD — DmmCalibMode.c:40：

```
// * 也是 dcms_mcu_topic_send() — 异步！
topicRet = dcms_mcu_topic_send(DCMS_CLIENT_DMM_CALIB_MODE_CMD,
                               (uint8_t*)&sendData[0], sizeof(sendData));
// 立即返回，不等待
```

两个都是异步发送。那 SOC 怎么拿到 MCU 的返回结果呢？通过回调 + 全局变量轮询。

异步模式的完整链路：发送 → 回调 → 轮询

以 DCMS_CLIENT_DMM_CALIB_MODE_CMD 为例，三步走：

```
// ===== 第1步：注册响应回调（初始化时，只执行一次） =====
// DiagCalib_Common.c:290
dcms_mcu_service_client_setup_resp_callback(
    DCMS_CLIENT_DMM_CALIB_MODE_CMD, // 我是 Client
    DiagCalib_DmmModeStatusCallback // 收到 ACK 时调这个
);

// ===== 第2步：收到 ACK 的回调 — 存到全局变量 =====
// DmmCalibMode.c:71-101
static uint8_t gDmmCalibModeStatus = DSAR_DMM_CALIB_ALLOW; // 全局变量

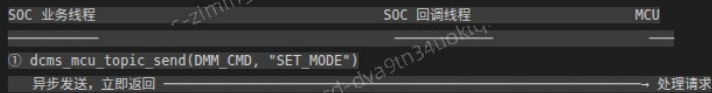
int32_t DiagCalib_DmmModeStatusCallback(const uint8_t* data, uint32_t len)
{
    // data 就是 MCU_Server 通过 set_ackdata 设置的 ACK 数据
    uint8_t cmd = data[0];
    if (cmd == DSAR_DMM_CALIB_MODE_SET) {
        gDmmCalibModeStatus = data[1]; // 存到全局变量
    } else if (cmd == DSAR_DMM_CALIB_MODE_QUERY) {
        gDmmCalibModeStatus = data[1]; // 同上
    }
    return 0;
}

// ===== 第3步：业务代码 — 发送后轮询全局变量 =====
// DiagCalib_HmiStatic.c:169-182 典型的 发送-等待-读取 模式
DiagCalib_DmmModeCmd(DSAR_DMM_CALIB_MODE_SET, DMM_EVENT_START_STATIC_CALIB);
// ↑ 上面是异步发送，立即返回

// 隔几个周期后...
DiagCalib_DmmModeCmd(DSAR_DMM_CALIB_MODE_QUERY, 0);
// ↑ 发查询命令，让 MCU 返回当前状态

dmmSetModeResult = DiagCalib_DmmModeStatusGet();
// ↑ 读全局变量 gDmmCalibModeStatus
// (这个变量已被 DiagCalib_DmmModeStatusCallback 更新过)
```

时序图：



```
set_ackdata({SET, SUCCESS})

② 继续往下执行其他代码... ③ DiagCalib_DmmModeStatusCallback()
    - 收到 ACK: data = {SET, SUCCESS}
    gDmmCalibModeStatus = SUCCESS → 自动发 ACK

④ 下一个周期:
    dcms_mcu_topic_send(DMM_CMD, "QUERY")
    异步发送 → set_ackdata({QUERY, SUCCESS})

⑤ 收到 ACK, 更新 gDmmCalibModeStatus

⑥ dmmSetModeResult = DiagCalib_DmmModeStatusGet()
    读到 gDmmCalibModeStatus == SUCCESS ✓
```

同步模式 — 如果你想直接拿到结果

如果想同步调用, 用 `dcms_mcu_topic_send_sync`:

```
// 同步版本 (伪代码, 这个服务实际上是异步的, 这里只是演示 API 用法)
uint8_t resp[2] = {0};
uint32_t resp_len = 0;

int32_t ret = dcms_mcu_topic_send_sync(
    DCMS_CLIENT_DMM_CALIB_MODE_CMD, // topic
    sendData, sizeof(sendData),      // 请求
    resp, &resp_len,                 // 响应 buffer
    500000                           // 超时 500ms
);
// 调用阻塞在这里, 直到 MCU 返回或超时
// 成功时 resp[0] 和 resp[1] 里已经有了返回值
// 不需要回调、不需要全局变量、不需要轮询
```

判断同步/异步的速查法

看 SOC 侧发请求的代码, 找到那个发送函数:

```
dcms_mcu_topic_send(topic, data, len)
→ 异步。只有 3 个参数, 没有响应 buffer 和 timeout
→ 附近通常有 dcms_mcu_service_client_setup_resp_callback 注册
→ 回调里通常会设置某个全局变量/标志位
```

```
dcms_mcu_topic_send_sync(topic, data, len, resp, &resp_len, timeout_us)
→ 同步。有 resp 和 timeout_us 参数
→ 不需要 resp_cbk
```

```
minidcos_service_call(name, &in, &out, retry, timeout_us)
→ 同步。有 out 参数和 timeout_us
```

一句话

不看 topic 配置, 看发送时用的函数名。 `dcms_mcu_topic_send` = 异步 (回调收结果), `dcms_mcu_topic_send_sync` 或 `minidcos_service_call` = 同步 (直接拿结果)。你图表里的两个都是 `dcms_mcu_topic_send`, 所以都是异步的。